MODULE 08 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

DynamoDB II – Reading, Writing and Capacity

Query against Scan, consistency, conditional writes, transactions, capacity modes and DAX.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 8 of 34

admin@sthithapragna.com

DynamoDB charges you for what it **reads**, not for what it returns. Almost every cost surprise comes from forgetting that.

The three read operations

OPERATION	WHAT IT READS
`GetItem`	One item, by complete primary key. The cheapest read there is
`Query`	Items under one partition key. Reads only the matching range
`Scan`	Every item in the table or index, page by page

ON THE EXAM

Query requires a partition key value. If your access pattern cannot supply one, the fix is an index, not a Scan.

Query in detail

- **KeyConditionExpression** selects what is read. Equality on the partition key, plus an optional sort key condition.
- **FilterExpression** discards items **after** they are read. It reduces the response, never the cost.
- **ProjectionExpression** chooses which attributes come back. It does not reduce read cost either.
- **ScanIndexForward: false** returns items in descending sort key order. The way to get 'most recent'.
- The partition key condition must be equality. No ranges, no begins_with.

> ON THE EXAM
>
> Only the key condition affects what you pay. Getting FilterExpression and KeyConditionExpression the wrong way round is a standard exam trap.

Query cost, worked

- A partition holds 1,000 items of 2 KB each. You query it with a filter matching 10 of them.
- DynamoDB reads all 1,000 items: 2,000 KB, which is 500 blocks of 4 KB.
- Eventually consistent, that is **250 RCUs** – to return 10 items.
- A sort key condition instead of a filter would read only the matching range.
- The 1 MB page limit also applies **before** filtering, so results arrive in pages.

> ON THE EXAM
>
> Same result set, wildly different cost. This is the single most useful calculation to be able to do quickly.

Scan, and when it is acceptable

- Reads every item, then applies any filter. Cost is proportional to **table size**, not results.
- Fine on genuinely small tables, or for one-off administrative work.
- **Parallel scan** splits the table into segments read concurrently. Faster, not cheaper.
- A parallel scan can consume all your capacity and throttle production traffic.
- Analytics over a whole table belongs in S3 or Athena, via an export.

> ON THE EXAM
>
> Parallel scan reduces wall-clock time and consumes capacity faster. If a scenario complains about cost, parallel scan is the wrong answer.

Pagination

- Query and Scan return at most **1 MB** per call, then stop.
- The response carries **LastEvaluatedKey** if more data exists.
- Pass it back as **ExclusiveStartKey** to fetch the next page.
- **Limit** caps items **read**, before filtering. So a filtered page can come back empty.
- No LastEvaluatedKey means you are done. An empty page does not.

> ON THE EXAM
>
> An empty page with a LastEvaluatedKey is normal, not a bug. Code that stops on the first empty result silently misses data.

Eventually or strongly consistent?

EVENTUALLY CONSISTENT

The default

- May not reflect a very recent write
- Costs half an RCU per 4 KB
- The only option on a GSI
- Reads can be served by any replica
- Right for the large majority of reads

STRONGLY CONSISTENT

Ask for it explicitly

- Always reflects all prior writes
- Costs a full RCU per 4 KB
- Set ConsistentRead to true
- Not available on a GSI, ever
- Higher latency, and less available

THE TIEBREAKER

Twice the price for the guarantee. Read-after-write on the same item is the case that needs it - and if the read goes through a GSI, it is simply impossible.

Capacity unit arithmetic

OPERATION	UNITS CONSUMED
Strongly consistent read	1 RCU per 4 KB, rounded up
Eventually consistent read	0.5 RCU per 4 KB, rounded up
Transactional read	2 RCU per 4 KB
Write	1 WCU per 1 KB, rounded up
Transactional write	2 WCU per 1 KB

ON THE EXAM

Reads round to 4 KB, writes round to 1 KB, and everything rounds up. A 0.5 KB write still costs a full WCU.

Provisioned or on-demand?

PROVISIONED

You set the capacity

- Cheapest for steady, predictable load
- Auto scaling adjusts within bounds
- Reserved capacity gives deeper discounts
- Throttles when you exceed it
- Needs capacity planning

ON-DEMAND

Pay per request

- No capacity planning at all
- Instant response to spikes
- Notably dearer per request
- Ideal for new or spiky workloads
- Still bound by per-partition limits

THE TIEBREAKER

Switch modes as often as once every 24 hours. On-demand removes planning, not physics - a hot key throttles in either mode.

Auto scaling, burst and reserved capacity

- **Auto scaling** tracks a target utilisation, typically 70%, between a floor and a ceiling you set.
- It reacts to CloudWatch alarms, so it takes **minutes**. It cannot absorb a sudden spike.
- **Burst capacity** banks up to 5 minutes of unused capacity, which covers short bursts.
- Burst capacity is best effort and can be consumed by background processes.
- **Reserved capacity** is a 1 or 3 year commitment for a large discount. Provisioned mode only.

> ON THE EXAM
>
> Auto scaling handles trends, burst capacity handles seconds, and neither handles a flash sale. That gap is what on-demand is for.

Writing items

- **PutItem** replaces the entire item. Attributes you omit are deleted.
- **UpdateItem** modifies named attributes and creates the item if it is absent.
- Update actions: **SET**, **REMOVE**, **ADD**, **DELETE**.
- **ADD** on a number is an **atomic counter**. Concurrent increments never lose one.
- **ReturnValues** can hand back the item before or after the change, saving a second read.

> ON THE EXAM
>
> The PutItem behaviour surprises people. Reading an item, changing one field and putting it back is how concurrent updates get lost.

Conditional writes

- **ConditionExpression** applies the write only if the condition holds.
- If it fails, you get **ConditionalCheckFailedException** and nothing is written.
- **attribute_not_exists(pk)** turns PutItem into insert-if-absent, preventing overwrites.
- **attribute_exists(pk)** turns it into update-only.
- A failed condition still costs capacity. You paid for the check.

> ON THE EXAM
>
> Conditional writes are how idempotency is implemented on DynamoDB. Module 5's idempotency key pattern is exactly this.

Expression attribute names and values

- **#name** placeholders substitute attribute names. **:value** placeholders substitute values.
- DynamoDB has **reserved words**, including name, status, size, timestamp and count.
- Using one directly raises a ValidationException, which reads confusingly.
- ExpressionAttributeNames sidesteps the whole problem: **#s** mapped to **status**.
- Values must always be placeholders. You cannot inline a literal in an expression.

> ON THE EXAM
>
> A ValidationException on a query that looks correct is very often a reserved word. Status and name catch nearly everyone once.

Optimistic locking

- Keep a **version** attribute on the item and increment it on every write.
- Condition the write on the version you read. If it moved, someone else got there first.
- The application then re-reads and retries, or reports a conflict to the user.
- No locks are held, so nothing blocks and nothing deadlocks.
- Use an atomic counter instead when the update is a pure increment. It never conflicts.

> ON THE EXAM
>
> Optimistic locking is for read-modify-write on a whole item. Atomic counters are for arithmetic. Choosing the wrong one shows up as either lost updates or needless retries.

Batch and transaction limits

OPERATION	LIMITS AND BEHAVIOUR
`BatchGetItem`	Up to 100 items, 16 MB. Reads in parallel across tables
`BatchWriteItem`	Up to 25 puts or deletes, 16 MB. No updates, no conditions
`TransactGetItems`	Up to 100 items. All-or-nothing, 2x read cost
`TransactWriteItems`	Up to 100 actions. All-or-nothing, 2x write cost

ON THE EXAM

25 is the odd one out. Every other bulk operation caps at 100, because BatchWriteItem never received the increase the others did in 2022.

Batch or transaction?

BATCHWRITEITEM

Throughput

- Individual writes are independent
- Partial failure is normal
- Failures return in UnprocessedItems
- You must retry them yourself
- Standard capacity cost

TRANSACTWRITEITEMS

Correctness

- All succeed or none do
- Supports conditions per action
- Can span tables in one Region
- Fails whole if any condition fails
- Double the capacity cost

THE TIEBREAKER

Batch is a convenience wrapper around many independent writes. A transaction is an atomicity guarantee, and you pay double for it.

DynamoDB Streams

- An ordered log of item-level changes, retained for **24 hours**.
- View types: **KEYS_ONLY**, **NEW_IMAGE**, **OLD_IMAGE**, **NEW_AND_OLD_IMAGES**.
- Ordered per partition key, and read by Lambda through an event source mapping.
- Powers replication, aggregation, search indexing and audit trails.
- 24 hours is the whole retention. Miss the window and the change is gone.

> ON THE EXAM
>
> NEW_AND_OLD_IMAGES is what you need to compute what actually changed. KEYS_ONLY tells you only that something did.

Time to Live

- Name a Number attribute holding a **Unix epoch in seconds**. Not milliseconds.
- **Deletion consumes no write capacity.** It is genuinely free.
- Deletion is best effort, **typically within 48 hours** of expiry. It is not immediate.
- So expired items remain readable, and still cost RCUs, until removed.
- TTL deletions appear in Streams as REMOVE events, which is how you archive before losing them.

> ON THE EXAM
>
> Two traps: milliseconds instead of seconds means items never expire, and queries must filter out expired items themselves during the deletion window.

DynamoDB Accelerator

- A fully managed, in-memory cache in front of DynamoDB. Microsecond reads.
- **API-compatible**, so adopting it is a client change, not a code rewrite.
- Two caches: an **item cache** for GetItem, and a **query cache** for Query and Scan.
- **Write-through**: writes go to DynamoDB and the cache together.
- Runs in a VPC, and it **cannot serve strongly consistent reads**. Those pass through.

> ON THE EXAM
>
> DAX for read-heavy DynamoDB traffic; ElastiCache when you are caching something computed, or not from DynamoDB at all.

DAX or ElastiCache?

CHOOSE DAX WHEN

Caching DynamoDB itself

- The data source is DynamoDB
- You want no application changes
- Read-heavy, repetitive access
- Write-through keeps it fresh
- Microsecond reads are the goal

CHOOSE ELASTICACHE WHEN

Caching anything else

- Caching computed or aggregated results
- The source is RDS or an external API
- You need data structures like sorted sets
- You control invalidation yourself
- Also used for sessions and rate limiting

THE TIEBREAKER

If the scenario says 'reduce DynamoDB read latency with minimal code change', it is DAX. Anything requiring lists, counters or leaderboards is ElastiCache.

Choosing a data store

- **Ad hoc queries** you cannot predict. DynamoDB needs the access pattern up front.
- **Complex joins and aggregations** across entities. That is a relational problem.
- **Full-text search.** Use OpenSearch, fed from DynamoDB Streams.
- DynamoDB wins on **predictable single-digit millisecond** access at any scale.
- It also wins when you want no servers, no patching and no connection pool.

> ON THE EXAM
>
> An exam scenario describing reporting, analytics or arbitrary filtering is usually steering you to Aurora, Athena or OpenSearch, not to a bigger DynamoDB table.

Errors and retries

- **ProvisionedThroughputExceededException** means throttling. Retry with backoff.
- **ConditionalCheckFailedException** means your condition was false. **Do not retry blindly.**
- **ValidationException** is a malformed request. Retrying will never help.
- The AWS SDKs retry 5xx and throttling errors automatically, with exponential backoff and jitter.
- 4xx errors other than throttling are your bug, and the SDK will not retry them.

> ON THE EXAM
>
> The dividing line: 5xx and throttling are worth retrying, other 4xx errors are not. Conditional check failures need application logic, not a retry loop.

Two things worth recognising

- **PartiQL** gives DynamoDB a SQL-like query syntax over the same underlying operations.
- It does **not** add joins, and a badly written statement still becomes a Scan.
- **Global tables** replicate a table across Regions, active-active, in about a second.
- Conflicts resolve **last writer wins**, which is not always what you want.
- Global tables require Streams to be enabled on the table.

> ON THE EXAM
>
> PartiQL is a syntax, not a capability. Familiar SQL does not make an inefficient access pattern efficient.

Atomicity, at a glance

WHAT YOU NEED	WHAT TO USE
Increment a number safely	UpdateItem with ADD. An atomic counter
Write only if absent	PutItem with attribute_not_exists
Write only if unchanged	ConditionExpression on a version attribute
Several items, all or nothing	TransactWriteItems
Many independent writes, fast	BatchWriteItem, and retry UnprocessedItems

ON THE EXAM

Five different guarantees, five different tools. Reaching for a transaction when an atomic counter would do costs you double for nothing.

Two of these need arithmetic. Do it properly rather than estimating - the wrong answers are built from plausible miscalculations.

> A product catalogue table serves 2,000 reads per second. Each item is about 6 KB. The application tolerates data being a second or two out of date.
>
> The team is sizing provisioned read capacity and wants the lowest correct figure.

How many read capacity units should be provisioned?

A 1,000 RCUs

B 2,000 RCUs

C 4,000 RCUs

D 6,000 RCUs

◆ PAUSE AND DECIDE

How many read capacity units should be provisioned?

A 1,000 RCUs

Halves correctly but forgets the rounding. 6 KB is two 4 KB blocks, not one, so this is exactly half what is needed.

B 2,000 RCUs

6 KB rounds up to two 4 KB blocks. Eventually consistent costs 0.5 RCU per block, so 1 RCU per read. 2,000 reads per second needs 2,000 RCUs.

C 4,000 RCUs

Correct rounding, but priced as strongly consistent. The scenario allows stale data, so this overpays by exactly double.

D 6,000 RCUs

Treats each kilobyte as one unit. No rounding rule in DynamoDB works that way.

THE ARITHMETIC

6 KB rounds up to 8 KB, which is 2 blocks. Eventually consistent halves each block to 0.5, giving 1 RCU per read. 2,000 reads per second x 1 = 2,000 RCUs.

> A ticketing application must reserve a seat and decrement the remaining inventory for that event. Both live in the same DynamoDB table. If either write fails, neither may be applied, or the system will sell a seat it does not have.
>
> Seats are also occasionally released. The team needs a guarantee, not best-effort behaviour.

How should the developer implement the reservation?

A. Use BatchWriteItem with both the seat item and the inventory item

B. Use two UpdateItem calls in sequence and roll back manually if the second fails

C. Use TransactWriteItems with a condition that the seat is unreserved and inventory is above zero

D. Use UpdateItem with an atomic counter on the inventory item

◆ PAUSE AND DECIDE

How should the developer implement the reservation?

A BatchWriteItem

Not atomic. Individual writes can fail independently, which is exactly the outcome the scenario forbids. It also cannot take conditions.

B Sequential updates with manual rollback

The rollback itself can fail, and the process can die between the two writes. This is what transactions exist to replace.

C TransactWriteItems with conditions

All-or-nothing across both items, with per-action conditions preventing double booking and negative inventory. Costs double, which the guarantee justifies.

D Atomic counter

Handles the inventory decrement safely, and does nothing about the seat item. Half the problem, and inventory could go negative.

THE SIGNAL

'If either fails, neither may be applied' is the definition of a transaction. When a scenario states that requirement, it has named the answer.

A session table stores about 40 million active sessions. Sessions expire after 24 hours. A nightly Lambda function currently queries expired sessions and issues DeleteItem for each one, consuming roughly 8 million WCUs a night and often being throttled.

The team wants expired sessions removed without that write cost, and wants an archive record kept in Amazon S3 for compliance.

What should the team implement?

A Enable TTL on an expiry attribute, and process the resulting REMOVE events from DynamoDB Streams into S3

B Enable TTL on an expiry attribute and export the table to S3 nightly

C Replace DeleteItem with BatchWriteItem, deleting 25 sessions per call

D Switch the table to on-demand capacity so the deletes are never throttled

◆ PAUSE AND DECIDE

What should the team implement?

A TTL plus Streams to S3

TTL deletion consumes no write capacity at all, and TTL deletions surface in Streams as REMOVE events carrying the old image - which is exactly the archive record.

B TTL plus nightly export

TTL solves the cost, but an export captures what still exists. Sessions deleted between exports are never archived.

C BatchWriteItem

Fewer API calls, identical capacity cost. Batching saves network round trips, never write units.

D On-demand mode

Stops the throttling by paying for all 8 million writes instead. It makes the bill worse, not better.

THE SIGNAL

Two requirements: no write cost, and an archive. TTL alone gives the first. Streams with the old image gives the second, on the same mechanism.

Monitoring a table

- **ConsumedReadCapacityUnits** and **ConsumedWriteCapacityUnits** against what you provisioned.
- **ThrottledRequests**, plus **ReadThrottleEvents** and **WriteThrottleEvents** per index.
- **SuccessfulRequestLatency**, split by operation, for the DynamoDB side of your latency.
- **CloudWatch Contributor Insights** names the most accessed and most throttled **keys**.
- Metrics are per minute. Second-level throttling can hide inside a healthy-looking minute.

> ON THE EXAM
>
> Contributor Insights is the tool that turns 'the table is throttling' into 'this specific partition key is throttling'. Reach for it before changing capacity.

A dashboard reads the same few thousand configuration items from DynamoDB on nearly every page load, at about 30,000 reads per second. The DynamoDB bill is dominated by these reads, and p99 latency of 9 ms is now the slowest part of the page.

The configuration changes a handful of times per day. The team wants a fix that does not require rewriting the data access layer, and reads must reflect a change within a second or two.

What should the team implement?

A. Add a global secondary index on the configuration attributes

B. Put Amazon ElastiCache for Redis in front of DynamoDB with a 60-second TTL

C. Switch the reads to strongly consistent so they are served faster

D. Deploy a DAX cluster and point the existing client at the DAX endpoint

◆ PAUSE AND DECIDE

What should the team implement?

A Add a GSI

Indexes create new access patterns. The access pattern here already works - it is simply hot. A GSI adds cost and changes nothing.

B ElastiCache with a 60-second TTL

The strongest wrong answer. It would cut cost and latency, but it needs cache-aside code in the data layer, and a 60-second TTL breaks the two-second freshness requirement.

C Strongly consistent reads

Doubles the read cost and increases latency. It is the opposite of both requirements.

D DAX at the existing endpoint

API-compatible so the data layer is unchanged, microsecond reads, and write-through keeps it fresh on every change rather than on a timer.

THE SIGNAL

Compare with the DAX or ElastiCache tiebreaker. 'No rewrite' and 'fresh within seconds' both point at DAX - write-through beats a TTL for freshness.

What to carry forward

- You pay for what is **read**. Filters and projections shrink the response, never the bill.
- Reads round to **4 KB**, writes to **1 KB**. Eventually consistent is half price. Transactions are double.
- **BatchWriteItem is 25 and not atomic.** Transactions are 100 and all-or-nothing.
- TTL is free but best-effort, typically within 48 hours, and shows up in Streams.
- **DAX for DynamoDB, ElastiCache for everything else.** DAX cannot do strongly consistent reads.

> ON THE EXAM
>
> That completes DynamoDB. Next: Amazon S3 for developers - presigned URLs, multipart uploads, conditional writes and event notifications.